

Topic 2: Introduction to Control

First and second order systems

Prof. Gerardo Flores

Robotics and Automation (CSCE 3345 - SENG 3340)

TAMIU



Part 1

First-order systems

The simplest system there is

Start with a plant that has no dynamics of its own. Whatever we command, that is the velocity:

$$\dot{x} = u$$

This is not a toy: a robot commanded in velocity mode looks exactly like this from the outside.

We want x to reach a target x_d . Push against the error in proportion to its size:

$$u = -k(x - x_d), \quad k > 0.$$

The closed loop, written in the error

Substitute the controller into the plant:

$$\dot{x} = -k(x - x_d).$$

Now define the tracking error and differentiate it. Since x_d is constant, $\dot{e} = \dot{x}$:

$$e = x - x_d \implies \dot{e} = -k e$$

One equation, and it is the simplest differential equation in engineering. Everything we do for the rest of the semester is an attempt to reach this same equation for harder plants.

Solving it by hand

Step 1. Separate the variables:

$$\frac{de}{dt} = -ke \quad \implies \quad \frac{1}{e} de = -k dt.$$

Step 2. Integrate both sides:

$$\int \frac{1}{e} de = \int -k dt$$

$$\ln e = -kt + C.$$

Step 3. Exponentiate, and write $A = e^C$:

$$e(t) = A e^{-kt}.$$

Step 4. At $t = 0$ we get $A = e_0$, so

$$e(t) = e_0 e^{-kt}.$$

Step 5. Undo the change of variable, $x = e + x_d$:

$$x(t) = x_d + (x_0 - x_d) e^{-kt}$$

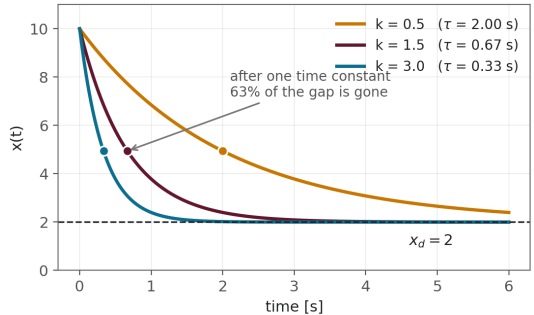
What the solution tells us

$$x(t) = x_d + (x_0 - x_d) e^{-kt}$$

The state reaches the target for any $k > 0$, from any initial condition, and it never overshoots.

The gain sets one thing only: the speed. The time constant is $\tau = 1/k$, and after about 5τ the error is under one percent.

So far, control looks easy.



Three gains, same target, same shape. Only the clock changes.

A real plant is not an integrator

Motors, tanks, circuits and vehicles carry their own dynamics. The honest model has a term in x on the right-hand side:

$$\dot{x} = ax + bu, \quad a = 3, \quad b = 2, \quad x_d = 8.$$

Here a is positive, so with $u = 0$ the state runs away on its own. The plant is not waiting for us, it is working against us.

Apply the same controller as before, $u = -k(x - x_d)$, and collect terms:

$$\dot{x} = (a - bk)x + bkx_d$$

Reading the new closed loop

Compare it against the integrator, where the closed loop was simply $\dot{e} = -ke$.

The coefficient multiplying x is now $a - bk$, not $-k$. Feedback moves the eigenvalue, but it starts from a , not from zero.

And a constant term $bk x_d$ is sitting on the right-hand side. It never disappeared.

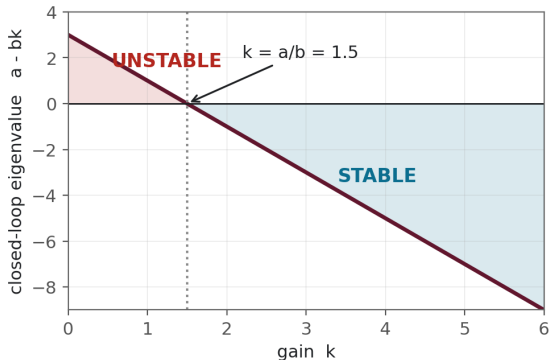
Each of those two differences costs us a result. We take them one at a time.

Result 1: feedback alone can still blow up

The closed loop is stable only when its eigenvalue is negative:

$$a - bk < 0 \iff k > \frac{a}{b} = 1.5$$

On the integrator every positive gain worked. Here there is a threshold, and below it the state runs to infinity even though the controller is present and the code is correct.



Gain is no longer a knob for speed alone. It decides whether there is a stable response at all.

Result 2: stable is not the same as correct

Pick $k > 1.5$ so the system settles.

Where does it settle? Set $\dot{x} = 0$:

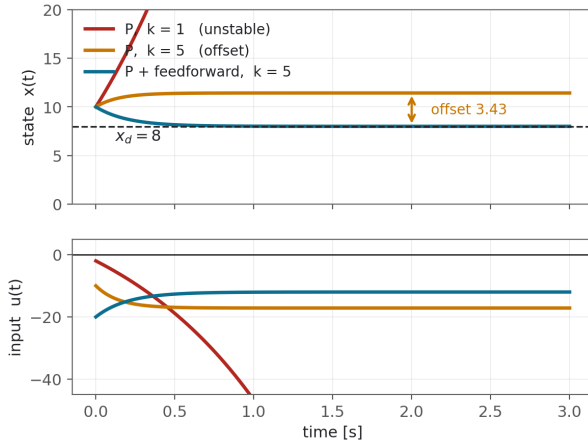
$$x^* = \frac{bk x_d}{bk - a}, \quad e^* = \frac{a x_d}{bk - a}$$

The numerator carries a , the term the controller never cancelled. On the integrator a was zero and the error vanished. Here it does not.

k	x^*	e^*
2	32.00	24.00
3	16.00	8.00
5	11.43	3.43
20	8.65	0.65
100	8.12	0.12

The error vanishes only as $k \rightarrow \infty$, and no real actuator goes there.

The experiment



P, $k = 1$. Unstable. The gain sits below $a/b = 1.5$.

P, $k = 5$. Stable, but parked 3.43 above the reference, exactly as the formula predicts.

P plus feedforward. Exact. The error decays as e^{-kt} , just like the integrator did.

Look at the lower panel too. The controller that tracks best is not the one that works hardest.

The fix, step by step

Step 1. Define the error:

$$e = x - x_d.$$

Step 2. Differentiate it, and rewrite the plant in error coordinates using $x = e + x_d$ and $\dot{x}_d = 0$:

$$\dot{e} = \dot{x} = a(e + x_d) + b u = a e + b u + a x_d.$$

With $a = 3$, $b = 2$ and $x_d = 8$ this is

$$\dot{e} = 3e + 2u + 24$$

Solving for the controller

Step 3. Impose the error dynamics we want, and set the two expressions equal:

$$\dot{e} = -k e \quad \implies \quad 3e + 2u + 24 = -k e.$$

Step 4. Solve for u , the only unknown left. From
 $2u = -k e - 3e - 24$,

$$u = \frac{1}{2}(-k e - 3e - 24)$$

In general $u = \frac{1}{b}(-ke - ae - ax_d)$. With $x = e + x_d$ this is
 $u = \frac{1}{b}(-ax - k(x - x_d))$: feedback, plus a term that cancels ax .

Code 1.1 Three control laws on one plant

```
def control_law(x, law, k):  
    e = x - x_d  
    if law == "p":  
        return -k * e                # proportional only  
    return (1/b) * (-a*x - k*e)      # proportional + feedforward  
  
def dynamics(t, state, law, k):  
    x = state[0]  
    u = control_law(x, law, k)  
    return [a*x + b*u]  
  
sol = solve_ivp(dynamics, [0, T], [x0], t_eval=t, args=(law, k))  
u = control_law(sol.y[0], law, k)    # rebuild u AFTER integrating
```

Laboratory: four tasks

Write down your prediction before you run anything.

Task 1. Set $a = 0$, $b = 1$ and check that you recover $x_d + (x_0 - x_d)e^{-kt}$ exactly.

Task 2. Back to $a = 3$, $b = 2$. Run $k = 1, 1.4, 3$. Compute the sign of $a - bk$ first.

Task 3. Compare each final value against x^* , then switch on feedforward.

Task 4. Open `controlTracking.py` and compare the two laws there. The lagging run has a steady error of amplitude $A/\sqrt{1+k^2}$. Verify it.

A reference that moves

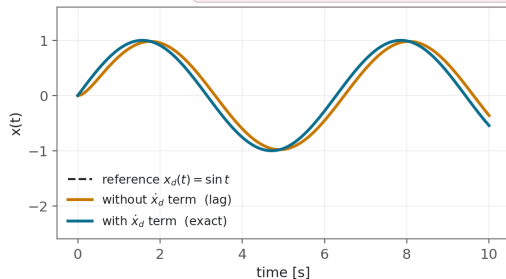
Let the target be $x_d(t) = \sin t$. Now $\dot{x}_d = \cos t$ is not zero, and it belongs in the error equation:

$$\dot{e} = ax + bu - \dot{x}_d$$

$$u = \frac{1}{b}(-ax - ke + \dot{x}_d)$$

Drop the $\cos t$ term and the error no longer decays to zero. It becomes a sinusoid of amplitude $1/\sqrt{1+k^2}$, so the robot follows the right shape and arrives late.

py controlTracking.py



Both controllers are stable. Only one of them is on time.

Where this goes next

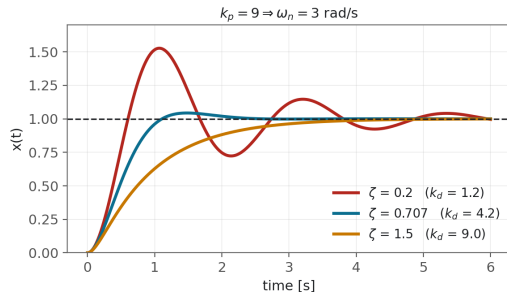
A robot joint is not first order either. Take $\ddot{x} = u$ and add derivative feedback:

$$u = -k_p(x - x_d) - k_d \dot{x}$$

$$\ddot{x} + k_d \dot{x} + k_p x = k_p x_d.$$

Two gains, and for the first time the response can overshoot and oscillate. One gain sets the speed, the other the damping.

In the plane nothing changes: with $p \in \mathbb{R}^2$ the error becomes a vector and k becomes a matrix.



Same natural frequency in all three runs.

Only the damping changes.

Part 1: what to remember

1. On $\dot{x} = u$, every positive gain works and the error decays as e^{-kt} .
2. On $\dot{x} = ax + bu$, the gain must clear a threshold: $k > a/b$.
3. Above the threshold it is stable, but it parks at the wrong value.
4. Feedforward recovers $\dot{e} = -ke$, and pays with the model.

Next: what changes when the plant is second order.

Part 2

The double integrator

One derivative further

So far every plant has been first order. A robot joint is not: Newton gives acceleration, not velocity, so the model of a mass driven by a force is

$$\ddot{x} = u$$

As a state-space system with position and velocity as the two states:

$$\dot{x} = v, \quad \dot{v} = u, \quad X = \begin{bmatrix} x \\ v \end{bmatrix}.$$

Two states, one input, and no way to act on x directly.

Why this plant is worth a whole session

It is the local model of almost every mechanical system you will control: a joint, a cart, a quadrotor axis, a positioning stage. Cancel the mass, the gravity and the friction, and what is left underneath is $\ddot{x} = u$.

It is also the first plant in this course that is not stable and not unstable. With $u = 0$ the state does not converge and does not explode. Give the mass a push and it coasts forever.

That middle ground is exactly where proportional control breaks down, and the reason is worth seeing before we fix it.

The obvious thing to try

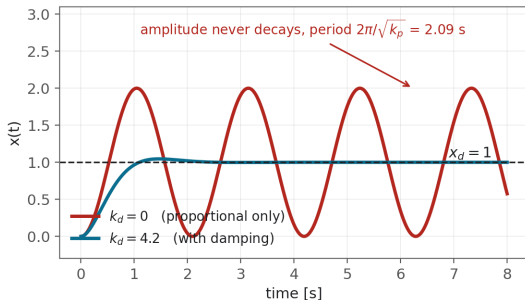
Apply what worked before,
proportional feedback on the position
error:

$$u = -k_p (x - x_d),$$

which turns $\ddot{x} = u$ into

$$\ddot{e} + k_p e = 0$$

A mass on a spring with no friction. It
oscillates around the target and never
arrives.



Raising k_p only makes it oscillate faster. There
is no gain that fixes this.

Why it fails, in one line

Look at the eigenvalues. The characteristic equation of $\ddot{e} + k_p e = 0$ is

$$\lambda^2 + k_p = 0 \implies \lambda = \pm j\sqrt{k_p}$$

They sit exactly on the imaginary axis. The real part is zero, so nothing decays.

Recall the pattern from last session: the error decays like $e^{\lambda t}$, and we need $\text{Re}(\lambda) < 0$. Proportional feedback can move these eigenvalues up and down the imaginary axis, but it cannot move them left. We need a second knob.

Adding the missing term

The second state is sitting right there, unused. Feed it back too:

$$u = -k_p (x - x_d) - k_d v.$$

That adds a first-derivative term, which is exactly friction:

$$\ddot{e} + k_d \dot{e} + k_p e = 0, \quad \lambda = \frac{-k_d \pm \sqrt{k_d^2 - 4k_p}}{2}$$

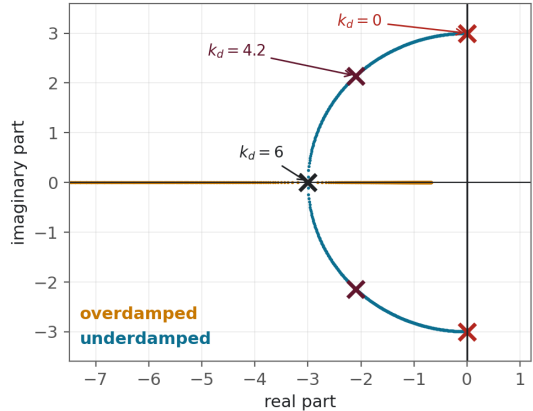
With both gains positive the error goes to zero. This is the PD controller, derived rather than announced.

Where the eigenvalues go

Fix $k_p = 9$ and increase k_d from zero. The pair starts on the imaginary axis at $\pm 3j$ and sweeps left along a circle.

At $k_d = 2\sqrt{k_p} = 6$ the two eigenvalues collide on the real axis. Past that they separate, and one of them crawls back toward the origin.

More damping is not always faster. That is the whole trade in one picture.



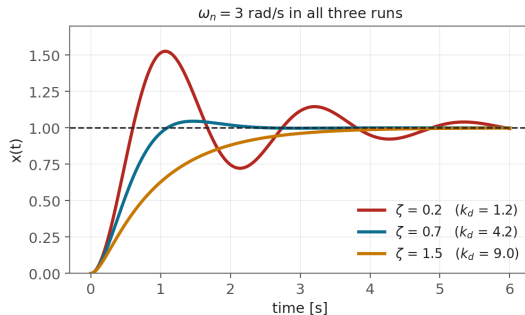
A better language than k_p and k_d

Compare the closed loop against the standard second-order form:

$$\ddot{e} + 2\zeta\omega_n \dot{e} + \omega_n^2 e = 0$$

$$k_p = \omega_n^2, \quad k_d = 2\zeta\omega_n$$

The gains are not the design variables. ω_n is how fast, ζ is how much it overshoots, and those are the two things a customer actually asks for.



Same ω_n , three damping ratios. Note that $\zeta = 1.5$ settles later than $\zeta = 0.7$.

The two numbers you need

ζ	overshoot M_p
0.2	52.7%
0.5	16.3%
0.7	4.6%
0.9	0.2%
1.0	0%

Overshoot depends on ζ alone, through $M_p = e^{-\pi\zeta/\sqrt{1-\zeta^2}}$. Settling time depends on the product $\zeta\omega_n$, which is the distance of the eigenvalues from the imaginary axis: $t_s \approx 4/(\zeta\omega_n)$ while $\zeta < 1$. Two requirements, two equations, two gains. 27/36

A design, start to finish

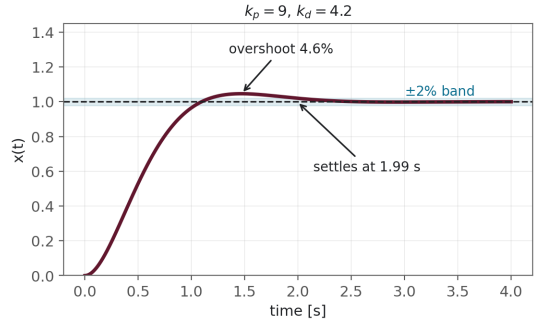
Requirement: overshoot under 5%,
settled within 2 seconds.

Step 1. Overshoot fixes the damping:
 $4.6\% < 5\%$, so $\zeta = 0.7$.

Step 2. Settling fixes the speed:
 $4/(\zeta\omega_n) \leq 2$ gives $\omega_n \geq 2.86$. Take
 $\omega_n = 3$.

Step 3. Convert to gains:
 $k_p = \omega_n^2 = 9$ and $k_d = 2\zeta\omega_n = 4.2$.

Step 4. Verify in simulation. You get
4.6% and 1.99 s.



Design on paper, verify in simulation, and only
then touch hardware.

Tracking a moving target

As in Part 1, following a trajectory needs more than the position:

$$x_d(t), \quad v_d(t) = \dot{x}_d, \quad a_d(t) = \ddot{x}_d.$$

Define the error states and differentiate twice, as in Step 2 of the recipe:

$$e = x - x_d, \quad \dot{e} = v - v_d, \quad \ddot{e} = u - a_d.$$

Choose u so that a_d cancels and a stable second-order error equation is left:

$$u = a_d - k_p(x - x_d) - k_d(v - v_d) \implies \ddot{e} + k_d\dot{e} + k_pe = 0$$

What the feedforward term buys

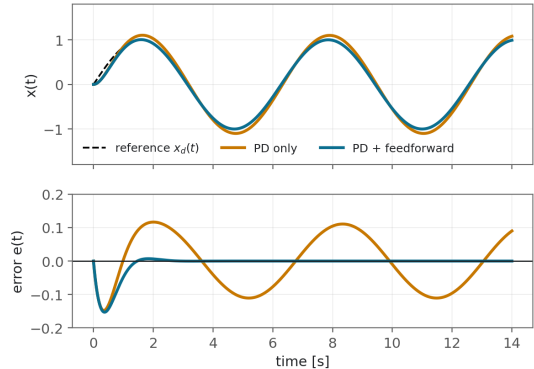
Drop a_d and the error equation is driven:

$$\ddot{e} + k_d \dot{e} + k_p e = -a_d(t).$$

For $x_d = A \sin \omega t$ the error settles into a sinusoid of amplitude

$$\frac{A \omega^2}{\sqrt{(k_p - \omega^2)^2 + (k_d \omega)^2}}$$

At $\omega = 1$ that is 0.111, and it never goes away.



Same gains in both runs. The only difference is one term.

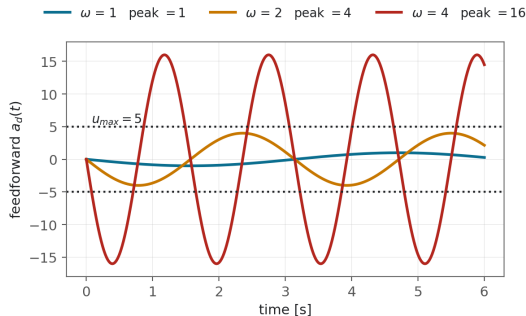
The trajectory you ask for has a price

For $x_d = A \sin \omega t$ the feedforward term is

$$a_d = -A\omega^2 \sin \omega t$$

The frequency enters squared.
Doubling the speed you ask for
quadruples the force you must
command.

With $u_{\max} = 5$ the $\omega = 4$ trajectory
is not executable. No controller fixes
that; the trajectory has to change.



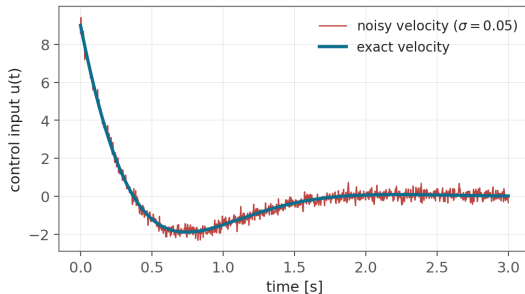
This is why trajectory generation and control
are designed together, not in sequence.

The derivative term is not free

The PD law needs v . On real hardware you usually measure position only, and differentiating a noisy signal amplifies the noise by k_d .

With $k_d = 4.2$ and a velocity estimate contaminated by $\sigma = 0.05$, the control signal carries a ripple of about 0.21. The motor hears every bit of it.

Two answers: filter the estimate, or build an observer. Both cost phase.



The state barely changes. The command does not.

Code 2.1 PD, feedforward, integral

```
def reference(t):                                # you need all three of them
    return (offset + A*np.sin(w*t), A*w*np.cos(w*t),
            -A*w**2*np.sin(w*t))

def control_law(t, pos, vel, integral):
    xd, vd, ad = reference(t)
    e, edot = pos - xd, vel - vd
    if MODE == "PD": u = -kp*e - kd*edot
    elif MODE == "PD+FF": u = ad - kp*e - kd*edot
    elif MODE == "PID+FF": u = ad - kp*e - kd*edot - ki*integral
    return u, e

def dynamics(t, state):
    pos, vel, integral = state
    u, e = control_law(t, pos, vel, integral)
    return [vel, u + d, e]                        # xdot=v, vdot=u+d, Idot=e
```

Laboratory: one warning, four tasks

Why the integral is a state and not a global variable

`solve_ivp` takes adaptive steps: it evaluates the right-hand side at trial times, rejects steps and moves backwards. A hand-written `I += e*(t - t_prev)` accumulates garbage. Carrying `I` as a third state lets the integrator do it correctly.

Task 1. `MODE = "PD"`. Check the error amplitude against the formula, then try $\omega = 3$.

Task 2. `"PD+FF"` with $d = 10$. Predict the offset first: only $-k_p e$ can supply $-d$.

Task 3. Keep $d = 10$, switch to `"PID+FF"`. What happened to the peak control effort?

Task 4. Set `U_MAX = 5` with $\omega = 3$. Which mode survives?

Where this goes next

Everything here was one axis. A robot arm is several, coupled through inertia and gravity, and the equations no longer separate.

The structure survives, though. Computed torque control cancels the coupling terms the way a_d cancelled the desired acceleration here, and leaves behind the same equation: $\ddot{e} + K_d\dot{e} + K_p e = 0$.

Three natural continuations:

An observer, so you can stop pretending you measure v .

An integral term, for the constant disturbances PD cannot reject.

LQR, which picks the gains by trading error against effort.

Part 2: what to remember

1. On $\ddot{x} = u$, proportional feedback alone oscillates forever.
2. The derivative term moves the eigenvalues left. That is why PD.
3. Design in ω_n and ζ , then convert: $k_p = \omega_n^2$, $k_d = 2\zeta\omega_n$.
4. Without a_d , tracking error is bounded but never zero.

Next session: what happens when you only measure position.